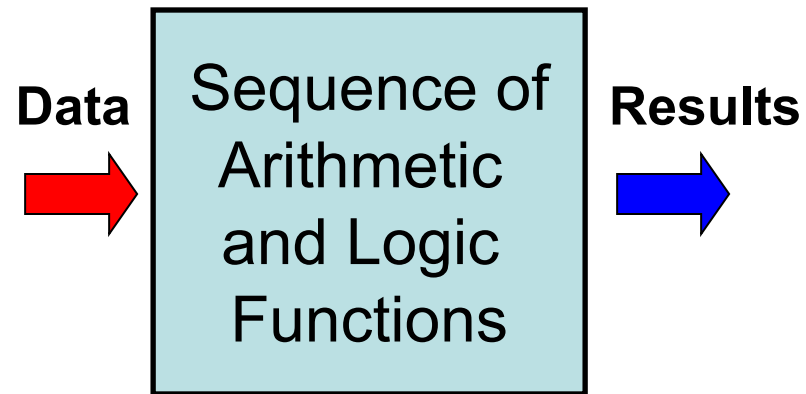


Chapter 2

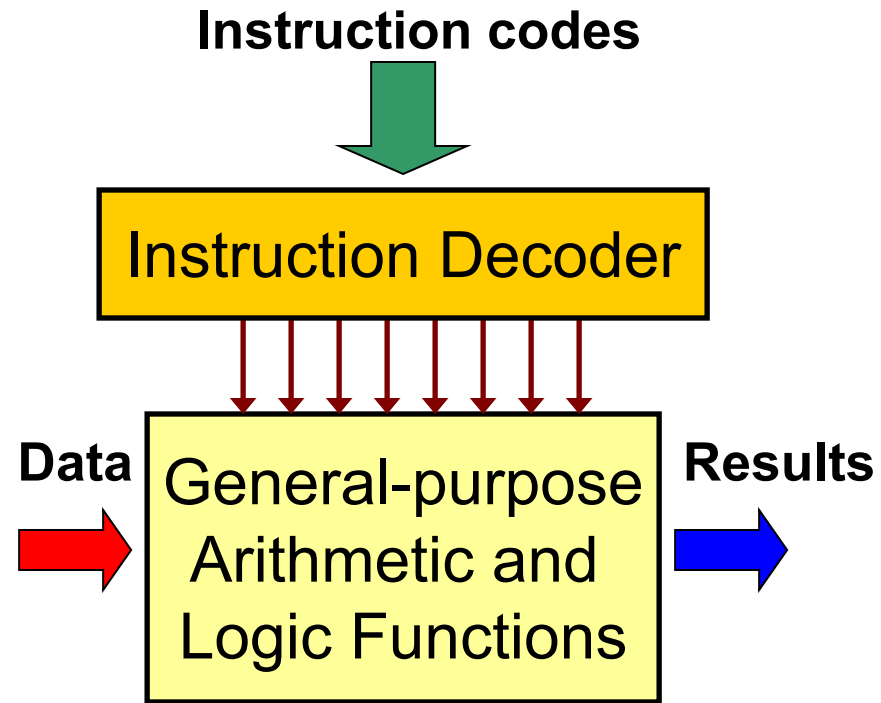
Data Organisation in Memory

Role of Memory in Computing

Approaches to Computing



**Programming in
Hardware**

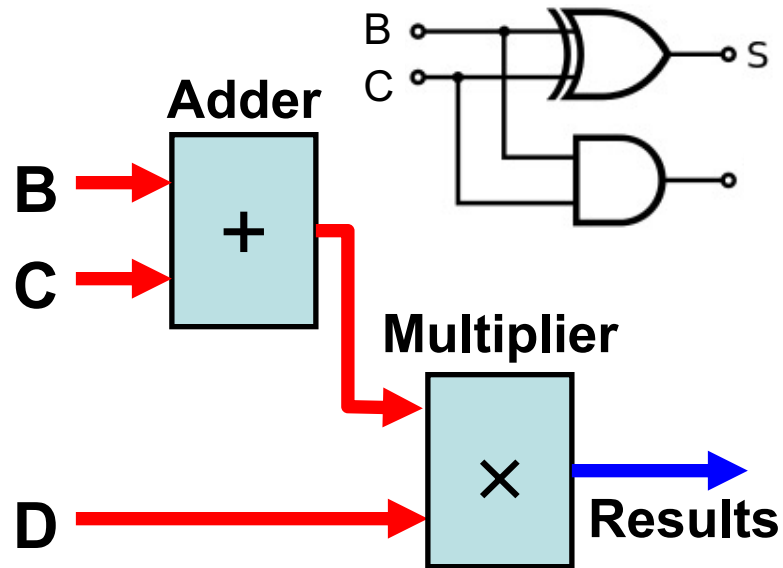


**Programming in
Software**

Approaches to Computing

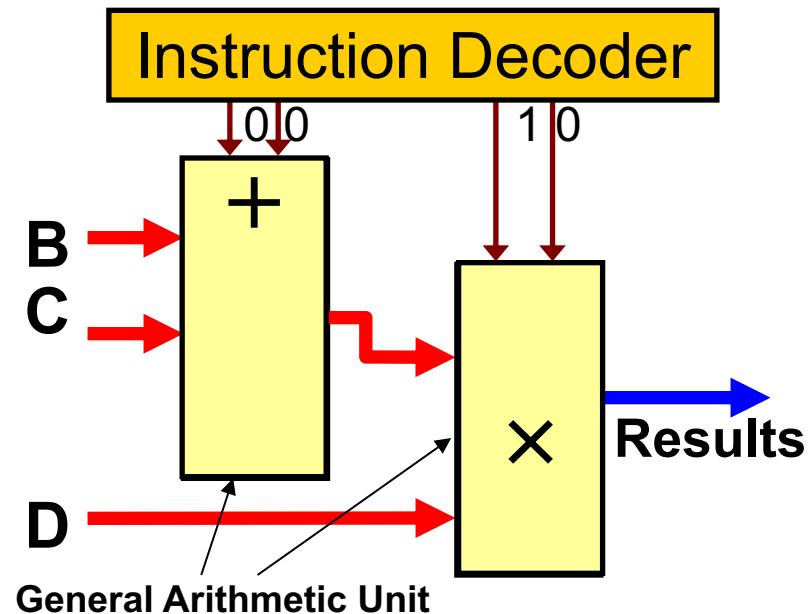
Implementing $A = (B+C) \times D$

Fast computation but
very inflexible



Programming in
Hardware

Slower and more complex
but easily programmable



Programming in
Software

Code, Data and Memory

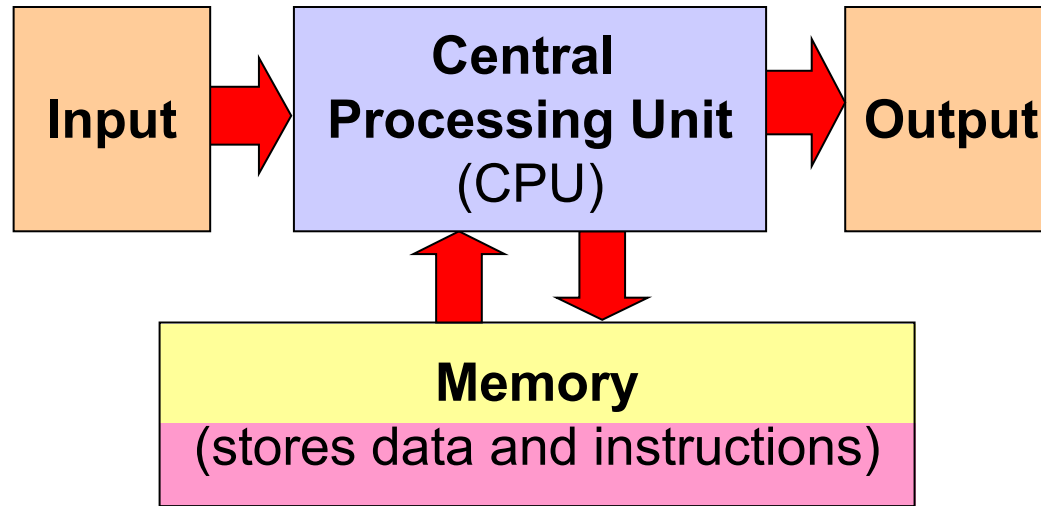
- What is code and what is data?
 - Code** is a sequence of instructions.
 - Data** are values these instructions operate on.
- What is the memory?
 - Its a sequential list of addressable storage elements for storing both instructions and data.

e.g. $B+C$

Address	Content	
0x000	+	Instruction
0x001		
0x002		
:	:	
:	:	
0x100	B	Data
0x101	C	
:	:	

Memory

The Stored Program Concept



- Most modern day computer design are based on von Neumann's stored program concept:
 1. Both data & instructions are stored in the same memory
 2. Contents of memory are addressable by location, without regard to data type
 3. Execution occurs sequentially (unless explicitly modified)

For the shown integer declaration, how will it be stored in memory assuming little-endian?

```
main()  
{  
    int x;    //declare an integer variable  
  
    x = 0x34126BFF;  
}
```

(A)

Address	Memory
N	0X34
N+1	0X12
N+2	0X6B
N+3	0XFF

(B)

Address	Memory
N	0X43
N+1	0X21
N+2	0XB6
N+3	0XFF

✓ (C)

Address	Memory
N	0XFF
N+1	0X6B
N+2	0X12
N+3	0X34

(D)

Address	Memory
N	0XFF
N+1	0XB6
N+2	0X21
N+3	0X43

For the shown memory content, what is the value of a possible negative short integer assuming little endian?

- A) 0x8FC5
- ✓ • B) 0xC58F
- C) 0XC538
- D) 0X835C

Address	Memory
0x1000	0X8F
0x1001	0XC5
0x1002	0X38
0x1003	

For a Pascal String, what is the maximum length of the string?

```
main()
{
    char s[4]="123"; //a string constant
}
```

- A) 256
- ✓ • B) 255
- C) 128
- D) No limit

Address	Content in Memory
0x0100	0x03
0x0101	0x31
0x0102	0x32
0x0103	0x33
0x0104	:
0x0105	:
0x0106	:

← 8 bits →

Pascal string

What is the size of the pointer shown in the code segment for the shown memory?

- A) 1 Byte
- ✓ • B) 2 Bytes
- C) 3 bytes
- D) 4 Bytes

```
main()
{
    char c ;    //char variable c
    char *ptr;  //char pointer ptr

    c = 'A' ;   //assign value 'A' to c
    ptr = &c;   //ptr gets address of c
}
```

Address	Content in Memory
0x0100	'A'
0x0101	:
0x0102	:
0x0103	:

← 16 bits → ← 8 bits →

What is the size of the pointer shown in the code segment for the shown memory?

- A) 8 bits
- B) 12 bits
- ✓ • C) 16 bits
- D) 24 bits

```
main()
{
    short int i ;
    Short int *ptr;

    i = 1500;
    ptr = &i;    //ptr gets address of c
}
```

Address	Content in Memory
0x100	:
0x101	:
0x102	:
0x103	:

← 12 bits → ← 8 bits →

For the shown struct, what is the correct padding for a 32-bit word machine?

```
struct data {
    char id[5];
    int val;
    char type;
}
rec1 r;
```

(A)

Address	Memory
0x000	r.id[0]
0x001	r.id[1]
0x002	r.id[2]
0x003	r.id[3]
0x004	r.id[4]
0x005	↑
0x006	r.val
0x007	↓
0x008	
0x009	r.type
0x00A	
0x00B	

(B)

Address	Memory
0x000	r.id[0]
0x001	r.id[1]
0x002	r.id[2]
0x003	r.id[3]
0x004	r.id[4]
0x005	
0x006	↑
0x007	r.val
0x008	↓
0x009	
0x00A	r.type
0x00B	

✓ (C)

Address	Memory
0x000	r.id[0]
0x001	r.id[1]
0x002	r.id[2]
0x003	r.id[3]
0x004	r.id[4]
0x005	
0x006	
0x007	
0x008	↑
0x009	r.val
0x00A	↓
0x00B	r.type

(D)

Address	Memory
0x000	↑
0x001	r.val
0x002	↓
0x003	
0x004	r.id[0]
0x005	r.id[1]
0x006	r.id[2]
0x007	r.id[3]
0x008	r.id[4]
0x009	r.type
0x00A	
0x00B	